

Tic-Tac-Toe Lab: Complete Sequential Coding Sessions

From absolute zero — HTML, then CSS, then JavaScript — up to the final project

Why this order: A browser builds a page the same way, every time: it reads the **HTML** to know *what exists* on the page, then the **CSS** to know *how it should look*, then the **JavaScript** to know *how it should behave*. Teaching in that order means every session lines up with something he can already see and touch from the session before — he's never learning a concept with nothing real to attach it to.

Same session format as before:

- **Builds on** — what he already knows going in
- **New concept** — the one new idea, no more
- **Where it lives in the code** — exact place to point at in your three files
- **How to teach it** — the actual script for the session
- **He does** — the hands-on task
- **Checkpoint** — one question to confirm it landed
- **Say the word** — vocabulary named *after* the activity

Total: 24 sessions across three parts, ending in the same finished game as before.

Part 1 — HTML: What Exists on the Page

Session 1 — A Page Is Just Labeled Boxes of Content

Builds on: Nothing. Day one, before any code editor even opens.

New concept: HTML doesn't make anything happen — it just describes what's on the page and what kind of thing each piece is (a heading, a paragraph, a button), using **tags**.

Where it lives in the code: The very top of `index.html`:

```
html

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tic-Tac-Toe Lab</title>
</head>
<body>
```

How to teach it:

1. Open `index.html` in the browser first — just look at the page, no code. Then open the file in the editor and scroll through it slowly, asking him to match pieces of code to things he can see on screen (e.g. *"Where do you think the words 'Tic-Tac-Toe Lab' are hiding in this file?"*).
2. Explain a tag comes in a pair: `<h1>` opens, `</h1>` closes, and everything between them is "inside" that box.
3. Point out `<!DOCTYPE html>` and `<html>`/`<head>`/`<body>` as the skeleton every HTML page starts with — `<head>` is information *about* the page (like a title bar), `<body>` is everything actually visible.

He does: In the browser tab, change the page's browser-tab title by editing the text inside `<title>...</title>`, reload, and watch the tab text change. First edit, instant proof it worked.

Checkpoint: *"If I asked you to find the part of the file that controls what shows up in the browser tab vs. what shows up on the page itself, could you point to both?"*

Say the word: *Tag, element* (a tag plus its content), *HTML*.

Session 2 — Text and Headings

Builds on: Session 1 (tags wrap content).

New concept: Different tags mean different kinds of text — headings (`<h1>`, `<h2>`, `<h3>`) for titles, `<p>` for regular paragraphs — and the browser styles them differently *by default*, before any CSS is even involved.

Where it lives in the code:

```
html

<h1 id="game-title">Tic-Tac-Toe Lab</h1>
...
<p class="eyebrow">Grade 10 coding challenge</p>
```

How to teach it:

1. Ask him to guess, just from the tag names, which text on the page each one produces (`<h1>` = the big bold title, `<p>` = the small label above it).
2. Have him notice `<h2>`, `<h3>` also appear elsewhere on the page (in the "Think Like a Game Designer" panel) — same idea, smaller.

He does: Add a brand-new `<p>` somewhere on the page with a sentence of his own, reload, and find it on the live page.

Checkpoint: "Why does `<h1>` show up bigger than `<p>` even though we haven't touched any styling yet?" (Browsers have default appearances for each tag — that's *not* CSS, that's just the tag's built-in identity.)

Say the word: Heading levels (`<h1>`–`<h6>`), paragraph.

Session 3 — Buttons and Interactive Elements

Builds on: Session 2 (different tags = different kinds of content).

New concept: Some elements exist specifically to be clicked or interacted with — `<button>` is the main one in this file.

Where it lives in the code:

```
html

<button id="newRoundButton" type="button">New round</button>
```

How to teach it:

1. Ask: "What's different about a `<button>` compared to a `<p>`, just from how it behaves on the page?" (Cursor changes, it looks clickable, it can be focused with Tab/keyboard.)
2. Point out all 9 game cells are also `<button>` elements — `<button class="cell" data-cell="0">`. This is a good early "aha": the whole board is secretly just a grid of buttons.

He does: Add a brand-new button anywhere on the page (it won't do anything yet — that's for later, JS sessions). This proves he understands buttons exist as HTML before behavior exists as JS — a subtle but important separation.

Checkpoint: "If I deleted every line of `script.js` right now, would the buttons still show up on the page?" (Yes — HTML draws the page regardless of whether JS exists. This distinction matters a lot later.)

Say the word: Interactive element.

Session 4 — Grouping Content: `<div>`, `<section>`, `<id>`, and `<class>`

Builds on: Sessions 1–3 (individual elements).

New concept: Elements can be grouped inside container elements, and every element can carry an **id** (a unique name, used once) or a **class** (a reusable label, used on many elements) to identify it for styling or code later.

Where it lives in the code:

```
html

<section class="game-panel" aria-labelledby="game-title">
  . . .
</section>
<aside class="thinking-panel" aria-labelledby="thinking-title">
```

and

```
html

<button class="cell" type="button" data-cell="0" aria-label="Top left square"><
```

How to teach it:

1. Ask him to find every place `id="something"` appears vs. `class="something"`. Build a simple rule together: *"If only one thing on the page should ever have this name, it's an id. If many things might share this description, it's a class."* Point at `class="cell"` — all 9 squares share it — versus `id="status"`, which only exists once.
2. Explain `<section>` and `<aside>` are just containers, like `<div>`, but with names that describe their *purpose* (main content vs. side content) — this is a light introduction to semantic HTML, without needing to go deep yet.

He does: Pick any group of related elements on the page and explain out loud why it's wrapped in a container, and whether each element inside uses an id or a class, and why.

Checkpoint: *"Could two different cells ever have the same class? Could two different elements ever share the same id?"* (Yes to the first, no to the second — reused vs. unique.)

Say the word: *Container element, id, class.*

Session 5 — Connecting the Three Files

Builds on: Everything in Part 1.

New concept: HTML, CSS, and JS are three separate files that get *linked together* — HTML doesn't automatically know they exist unless it's told to load them.

Where it lives in the code:

```
html
```

```
<link rel="stylesheet" href="styles.css">
...
<script src="script.js"></script>
```

How to teach it:

1. Ask: "Where in `index.html` does it mention the other two files by name?" Have him find both lines himself.
2. Have him comment out (or temporarily delete) the `<link>` line, reload, and see the page lose all styling but keep its content and button clicks — proof that HTML is independent of CSS.
3. Put it back, then temporarily remove the `<script>` line and show clicking cells now does nothing — proof that HTML is independent of JS too.

He does: Restore both lines and confirm everything works again. This "break it on purpose, see what depends on what" exercise is the real point of Part 1.

Checkpoint: "In your own words, what is each of the three files responsible for?" (This question is the whole reason Part 1 exists — make sure the answer is solid before Part 2 starts.)

Say the word: *Linking files*, and a first mention of *separation of concerns* (structure, style, and behavior living in different files on purpose).

Part 2 — CSS: How It Should Look

Session 6 — Selectors: Telling CSS What to Style

Builds on: Session 4 (id/class), Session 5 (files are linked).

New concept: CSS rules start with a **selector** that says *which* elements to style, followed by a block of properties in `{ }`.

Where it lives in the code:

```
CSS

.cell {
  border: 2px solid var(--line);
  ...
}
```

How to teach it:

1. Ask him to predict: "This selector starts with a dot. Where have we seen dots used like that before?" (Connect back to `class="cell"` from Session 4 — a `.` selector targets

a class.)

2. Show `#status` in the CSS and connect it to `id="status"` in the HTML — `#` targets an id, the same way `.` targets a class.
3. Change one property live (e.g. `border-radius` on `.cell`) and reload to see the effect.

He does: Find three different selectors in `styles.css`, and for each, find the matching HTML element(s) it targets.

Checkpoint: "If I wrote `.cell { color: purple; }`, how many elements on the page would turn purple?" (All 9 — every element with `class="cell"`.)

Say the word: *Selector, property, declaration* (a property + value pair).

Session 7 — The Box Model: Every Element Is a Box

Builds on: Session 6 (selectors target elements).

New concept: Every element on a page is secretly a rectangular box with **content**, **padding** (space inside the border), **border**, and **margin** (space outside the border).

Where it lives in the code:

CSS

```
.scoreboard div {  
  min-height: 74px;  
  border: 1px solid var(--line);  
  border-radius: 8px;  
  padding: 10px;  
}
```

How to teach it:

1. Open the browser DevTools, inspect one of the scoreboard boxes, and find the "box model" diagram DevTools shows (most browsers visualize this directly — a great non-code way to see the concept).
2. Change `padding` on `.scoreboard div` live and watch the space inside the box change, versus changing `margin` and watching the space *between* boxes change.

He does: Adjust padding and border-radius on `.cell` until the board looks noticeably different, then put it back (or keep a version he likes).

Checkpoint: "If a box feels too cramped from its neighbor, do you reach for padding or margin?" (Margin — space outside the box, between it and others.)

Say the word: *Box model, padding, margin, border.*

Session 8 — Colors and Reusable Values (CSS Variables)

Builds on: Session 6–7 (properties have values).

New concept: Instead of repeating a color code everywhere, CSS variables (custom properties) let you define a value once and reuse it — directly echoing the `MAX_PIECES` constant idea from JavaScript, but for styling.

Where it lives in the code:

```
CSS

:root {
  --accent: #0f8b8d;
  --x-color: #e84855;
  --o-color: #255f85;
}

...
.cell.x {
  color: var(--x-color);
}
```

How to teach it:

1. Ask: "`--x-color` is defined once at the top, in `:root`. How many places in the file do you think use `var(--x-color)`?" Have him search and count.
2. Change `--x-color` in one place at the top and reload — show every X on the board, and anywhere else that uses it, updates together.
3. Point out `body.theme-sunset` redefines *all* the same variable names with different colors — this is literally how the theme button works, and he can explain it fully now.

He does: Pick a new accent color, change `--accent` in `:root`, and see it ripple across buttons, borders, and highlights across the whole page.

Checkpoint: "Why is it better to define `--x-color` once instead of typing `#e84855` directly in five different CSS rules?" (One place to change it — same DRY idea from the JS sessions, applied to styling.)

Say the word: CSS custom property (CSS variable).

Session 9 — Arranging Things in a Row or Column (Flexbox)

Builds on: Session 7 (box model — boxes have size; now, how do boxes arrange next to each other).

New concept: `display: flex` arranges child elements in a row or column automatically, without manually positioning anything.

Where it lives in the code:

```
CSS

.title-row {
  display: flex;
  justify-content: space-between;
  gap: 16px;
  align-items: start;
}
```

How to teach it:

1. Point at `.title-row` in the HTML — it holds the title text and the theme button. Ask: *"How do you think this line makes the button sit on the far right, all the way across from the title, without anyone manually calculating pixel positions?"*
2. Change `justify-content` between `space-between`, `center`, and `flex-end` live, reloading each time, to build intuition by watching it move.
3. Point out `.controls` also uses flex — the two buttons at the bottom of the game.

He does: Add a third temporary button inside `.controls` and watch flexbox automatically make room for it in the row.

Checkpoint: *"What does the `gap` property do, and how is it different from margin?"* (Gap only adds space *between* items in the row/column, not around the whole group.)

Say the word: *Flexbox, flex container, flex item.*

Session 10 — Arranging Things in a Grid

Builds on: Session 9 (flexbox is 1-dimensional; grid is 2-dimensional — rows *and* columns at once).

New concept: `display: grid` arranges elements into a true rows-and-columns layout, which is exactly what a tic-tac-toe board needs.

Where it lives in the code:

```
CSS
```



```
.board {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 10px;  
  aspect-ratio: 1;  
}
```

How to teach it:

1. Ask: "There are 9 buttons in the HTML, in one long list, no rows or columns mentioned anywhere in the HTML. So how does the board end up looking like a 3x3 grid?" This is a great "the HTML doesn't know its own shape — CSS gives it one" moment.
2. Change `repeat(3, 1fr)` to `repeat(4, 1fr)` live and watch the layout reflow into an (uneven, since there are 9 items) 4-column grid — good proof that the number 3 here is genuinely doing the layout work.
3. Point out `.scoreboard` and the outer `.app` also use grid, at different scales (3 columns for scores, 2 columns for the whole page layout).

He does: Try changing `.board`'s grid to `repeat(2, 1fr)` and predict what the board will look like before reloading.

Checkpoint: "What's the actual difference between when you'd reach for flexbox versus grid?" (Flex for a single row/column of items; grid when you need both rows and columns to line up together.)

Say the word: *Grid, grid-template-columns, fr unit* (a "fraction of available space").

Session 11 — Responsive Design: Different Screens, Different Layouts

Builds on: Session 10 (grid layout), Session 9 (flex layout).

New concept: `@media` rules let CSS apply different styles depending on screen size, so the page adapts between desktop and mobile.

Where it lives in the code:

```
CSS  
  
@media (max-width: 820px) {  
  .app {  
    grid-template-columns: 1fr;  
  }  
}
```

How to teach it:

1. Open the browser's device toolbar (responsive mode) and resize the viewport live while watching the two-panel layout collapse into one column.
2. Point out the media query is really just a normal CSS rule (`.app { grid-template-columns: ... }`) wrapped in a condition about screen width — connect this back to Session 4's JS conditionals (`if`) as the same *idea*, applied to CSS instead of logic.

He does: Add a new `@media` rule of his own — e.g., make `.cell` font size shrink further below 400px width — and test it in responsive mode.

Checkpoint: "What does `max-width: 820px` actually mean — does this rule apply above or below that width?" (Below/at — "max-width" means "while the screen is at most this wide.")

Say the word: *Responsive design, media query, breakpoint.*

Part 3 — JavaScript: How It Should Behave

(This part is the same conceptual sequence as before, renumbered to continue naturally from Part 2. Each session still points at real lines in `script.js`, but now he arrives already fluent in what the HTML and CSS around it are doing — so when a JS session touches `classList.add("win")`, he already knows exactly what `.win` does in the CSS, because he built that understanding in Session 6.)

Session 12 — Programs Run Top to Bottom, But Wait for You

New concept: Event listeners — code that waits for a click. **Code:** `cells.forEach((cell) => { cell.addEventListener("click", handleCellClick); }));` **Say the word:** *Event listener.*

Session 13 — Boxes That Hold Information

New concept: Variables. **Code:** `let currentPlayer = "X";` **Say the word:** *Variable.*

Session 14 — Lists of Things

New concept: Arrays and indexes. **Code:** `let board = ["", "", "", "", "", "", "", "", "", ""];` **Say the word:** *Array, index.*

Session 15 — Making Decisions

New concept: Conditionals. **Code:** `if (board[index] !== "" || gameOver) { return; }` **Say the word:** *Conditional.*

Session 16 — Reusable Instructions

New concept: Functions. **Code:** `function makeMove(index) { ... }` **Say the word:** *Function, parameter.*

New concept: Loops. **Code:** `cells.forEach((cell, index) => { ... })` **Say the word:** *Loop*.

Session 18 — Grouping Related Data

New concept: Objects. **Code:** `let scores = { X: 0, O: 0, Rounds: 0 };` **Say the word:** *Object, key, value*.

Session 19 — Small, Safe Feature: Draw Detection

New concept: Combining prior concepts into real logic (no new syntax).

Session 20 — Talking to the Page (Now With Real CSS Context)

New concept: Reading/writing the DOM, and reconnecting it explicitly to Part 2. **Code:**

`document.querySelector("#status"), cell.classList.add("win")` **How to teach it**

differently than before: Since he now knows CSS classes from Session 6, spend extra time on `classList.add("win")` specifically — have him find the matching `.cell.win` rule in `styles.css` himself and explain, in his own words, that JavaScript is *adding a class name*, and CSS is what actually *decides what that class looks like*. This is the single most important "how the three files cooperate" moment in the whole curriculum. **Say the word:** *DOM*.

Session 21 — Reading Input (Player Names Feature)

New concept: Reading `.value` from `<input>` elements — first time HTML he adds himself feeds directly into JS logic. **Say the word:** *Input value*.

Session 22 — Code That Runs on a Timer

New concept: `setInterval` / `setTimeout`. **Say the word:** *Asynchronous*.

Session 23 — Teaching the Computer to "Think" (Simple AI Opponent)

New concept: Decision-making algorithms, reusing `findWinningLine()`. **Say the word:** *Algorithm*.

Session 24 — Final Project: Ship It

Same final feature set as before — names, draw detection, timer, AI opponent, and a polish item of his choosing — but now he can genuinely explain and modify *all three files*, not just the JavaScript. His final walkthrough (Session 10 of the original plan) should now include: which parts of the feature are HTML, which are CSS, and which are JS, for every single feature he built.

Quick Reference: The Full 24-Session Sequence

Part	#	Session	Concept
HTML	1	Tags & structure	<code><html></code> , <code><head></code> , <code><body></code>
HTML	2	Text & headings	<code><h1></code> , <code><p></code>
HTML	3	Interactive elements	<code><button></code>
HTML	4	Grouping & naming	<code><div></code> , <code>id</code> , <code>class</code>
HTML	5	Linking files	<code><link></code> , <code><script></code>
CSS	6	Selectors	<code>.cell</code> , <code>#status</code>
CSS	7	Box model	padding, border, margin
CSS	8	Variables	<code>--x-color</code> , <code>var()</code>
CSS	9	Flexbox	<code>.title-row</code> , <code>.controls</code>
CSS	10	Grid	<code>.board</code> , <code>.scoreboard</code>
CSS	11	Responsive design	<code>@media</code>
JS	12	Event listeners	<code>addEventListener</code>
JS	13	Variables	<code>let currentPlayer</code>
JS	14	Arrays	<code>board[index]</code>
JS	15	Conditionals	<code>if (...)</code>
JS	16	Functions	<code>function makeMove()</code>
JS	17	Loops	<code>cells.forEach()</code>
JS	18	Objects	<code>scores.X</code>
JS	19	Combining logic	Draw detection
JS	20	DOM (+ CSS tie-in)	<code>classList.add("win")</code>
JS	21	Reading input	<code>.value</code>
JS	22	Timers	<code>setInterval</code>
JS	23	Simple algorithms	AI opponent

Part	#	Session	Concept
JS	24	Final project	Everything, integrated

Pacing note: Part 1 (HTML) moves fast — most kids clear 2 sessions per sitting since there's no logic yet, just vocabulary and structure. Part 2 (CSS) is where visual feedback makes it fun; expect 1 session per sitting since layout concepts (flex/grid especially) take longer to click. Part 3 (JS) follows the same pacing notes as the original plan — sessions 19 onward will each naturally stretch across 1-2 sittings.